Task 1 | Development and reflection phase

Implementation procedure and system placement | 17 October 2026

Repository URL to add after publishing: https://github.com//municipal-air-quality-batch-ingestion

A Python loader sits between the public source file and MongoDB; a separate API or BI layer can later serve station/time queries to the city dashboard. Docker Compose starts both services and waits for the database health check. An initialization script validates shared fields and adds a station/timestamp index.

The loader downloads the UCI archive, reads CSV rows incrementally, converts each row to a station-hour document, then writes batches of 1,000. Missing values are omitted and unknown measurement fields are preserved. The source time is interpreted as Asia/Shanghai. Deterministic IDs and `$setOnInsert` upserts make replay safe after partial writes; transient database errors receive up to five exponential-backoff retries.

Four unit tests pass for normalization, missing values, future fields, batch boundaries, and invalid batch size; Python compilation passes. A live Docker/MongoDB run was not possible in the authoring environment because Docker is unavailable. Before submission, run `docker compose up --build`, verify the record count and one station/time query, and replace the placeholder with the real repository URL.

The main lesson is that retry safety requires idempotent writes, not only exception handling. Add your own observations from the Docker run to the final personal reflection.

Reference: MongoDB PyMongo bulk writes, https://www.mongodb.com/docs/languages/python/pymongo-driver/current/crud/bulk-write/.